Open book exam. Duration: 1h30m First Midterm Exam ("Mini-Teste")

Group 1. *First*, *Follow* Sets and *Parsers* LL(k) (5 pts)

E → *E* + *T* | *T*
T → *T* * *F* | *F*
F → (*E*) | **NUM**

Consider the CFG[1] in the left. **NUM** is a terminal symbol representing numbers.

1.a) [1pt] Give the *First* and *Follow* sets for each grammar variable;

1.b) [2pts] Is the grammar LL(1)? Show the table for the *parser* LL(1);

1.c) [2pts] Modify the grammar in order it can be implemented as a top-down recursive parser with K=1 (*lookahead*).

Group 2. Lexical and Syntactic Analysis (6 pts)

In the context of the development of Java applications for smartphones, we intend to implement a parser of directives based on code regions with the intention to migrate (offload) them to a server. This language will be embedded in the Java code of the application using specific Java comments that represent directives. Those directives are preceded by "//$cloud" and, at the moment, there exist only directives related to tasks. The example below shows the use of two directives. The presented CFG represents the current version of the language.

```
   // app code
…
//$cloud task offload : in scalar A int, out arrayref B
double[][]
…
   //app code
…
//$cloud end task
…
   //app code
```

```
1.  S → //$cloud task Directive | //$cloud end task
2.  Directive → Action : Params
3.  Action → offload
4.  Params → Flow Data (, Flow Data)*
5.  Flow → in | out | inout
6.  Data → (scalar | classref | arrayref) IDENTIFIER Type
7.  Type → (int | float | double) ([])*
```

2.a) [0,5pt] Show the regular expression for the token IDENTIFIER (consider ∑={a,...,z,A,...,Z,0..9,_}).

2.b) [1pt] Show the DFA for the tokens present in the grammar rules in lines 1, 2, 4, e 5.

2.c) [1pt] Show the concrete syntax trees for the two directives in the example and considering the depicted CFG.

2.d) [0,5pt] Show a possible abstract syntax tree (AST[2]) for each concrete syntax trees of the question above.

2.e) [2pt] Show the necessary functions, considering the grammar rules of lines 1 to 4, and the respective pseudo-code to implement as a top-down recursive parsed the language defined by the grammar LL(1). Assume the existence of the lexical analyzer which outputs the sequence of tokens, the existence the global variable *token*, and the function *next()* which returns the next token in the sequence of tokens (in the beginning the variable *token* identifies the first token in the sequence).

2.f) [1pt] Suppose that we intend to modify the language in order that the variable *Action* can indicate unconditional "offload" but also can indicate cases where "offload" is based in rules like:

server.on > offload

and

server.off > onload

Indicate the modifications of the CFG to also deal with these rules.

Group 3. Semantic Analysis (5 pts)

Consider the segment of Java code of the example below.

---

[1] Do inglês: *Context-Free Grammar*

[2] Do inglês: *Abstract Syntax Tree*

Example

```
1.  public void medianandavg(int numArray[], double result[]) {
2.      int N = numArray.length;
3.      long sum = 0;
4.      for (int i = 0; i < N; i++) {
5.          sum += numArray[i];
6.      }
7.      long total = sum;
8.      double avg = total / N;
9.      int median = numArray[N/2];
10.     result[0] = median;
11.     result[1] = avg;
12. }
```

3.a) [2pts] Assuming that the variables used in the code can only be parameters or local variables, show a possible symbol table for this example indicating the information to be included in the descriptors.

3.b) [2pts] Show a high-level intermediate representation for the example, using as reference the high-level intermediate representation based on expression trees and presented in the classes of the course.

3.c) [1pt] Describe possible semantic verifications to be taken into account for the code in lines 8 and 9 and indicate how those verifications can be done.

Group 4. General (4 pts)

Comment the following sentences, indicating if they are true or false, and justifying your answers (if helpful include illustrative examples to help on justifying your answers):

4.a) [2pts] "The existence of left recursivity in CFG's make impossible the implementation of them.".

4.b) [2pts] "The only way to satisfy the precedence of operators in arithmetic and/or logic expressions is to ensure that the concrete syntax tree respect those precedencies".

(End.)